Topic 1: Process Synchronization, Mutexes, and Semaphores

Process synchronization coordinates the execution of concurrent processes to ensure consistent data.

A critical section is a segment of code where shared resources such as memory, files, or variables are accessed.

Mutual exclusion guarantees that only one process or thread executes within its critical section at any given time.

Race conditions occur when the outcome of execution depends on the non-deterministic scheduling of competing threads.

Semaphores are integer variables accessed only through two atomic operations: wait() also known as P(), and signal() also known as V().

A binary semaphore takes values 0 or 1 and functions identically to a mutex lock.

A counting semaphore has an unrestricted domain and is used to control access to a resource pool with multiple instances.

Common synchronization problems include the Producer-Consumer Problem, Readers-Writers Problem, and Dining Philosophers Problem.

Topic 2: Deadlock Characterization, Prevention, and Banker’s Algorithm

A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another.

Four Coffman conditions must hold simultaneously for a deadlock to arise:

1. Mutual Exclusion: At least one resource must be held in a non-shareable mode.

2. Hold and Wait: A process must be holding at least one resource and waiting to acquire additional resources.

3. No Preemption: Resources cannot be preempted; a resource can be released only voluntarily by the process holding it.

4. Circular Wait: A closed chain of processes exists such that each process holds resources needed by the next process.

Deadlock prevention eliminates at least one of the four necessary Coffman conditions.

Deadlock avoidance uses dynamically gathered information such as Dijkstra’s Banker’s Algorithm to ensure the system never enters an unsafe state.

A safe state guarantees that there exists a safe sequence of processes that can finish execution without deadlocking.

Topic 3: Virtual Memory, Paging, and Page Replacement Algorithms

Virtual memory separates user logical memory from physical RAM, allowing execution of processes larger than physical memory.

Paging divides logical memory into fixed-size blocks called pages, and physical memory into matching blocks called frames.

The Memory Management Unit (MMU) translates logical addresses into physical addresses using a Page Table.

The Translation Lookaside Buffer (TLB) is an associative hardware cache that drastically reduces memory lookup latency.

A page fault interrupt is triggered when a process references a page marked invalid or not resident in physical RAM.

Page replacement algorithms determine which frame to evict when all frames are occupied:

1. FIFO (First-In, First-Out): Simple, but suffers from Belady's Anomaly where adding frames can increase page faults.

2. LRU (Least Recently Used): Replaces the page unused for the longest time, highly optimal in practice.

3. Optimal Algorithm: Replaces the page that will not be used for the longest period in the future (theoretical benchmark).

Thrashing occurs when a computer spends excessive time swapping pages into and out of memory rather than executing instructions.